Übungsblatt 2: RISC-V-Registerbefehle

Papieraufgaben und schrittweises Prüfen mit Venus

Ziel und Arbeitsweise

Sie verfolgen Registerzustände, prüfen unmittelbare Konstanten und übersetzen zwischen Assemblertext und Befehlswort. Dafür genügen die Inhalte aus Vorlesung 1 und 2; C, Speicherbefehle und Funktionsaufrufe werden nicht vorausgesetzt.

Bearbeiten Sie Aufgaben 1–6 zunächst auf Papier, ohne Simulator. In Aufgabe 7 vergleichen Sie Ihre Vorhersage mit der Kursdistribution von Venus. Öffnen Sie die Musterlösung erst nach Ihrer eigenen Bearbeitung. Die **60 Punkte** der Papieraufgaben dienen der Selbstkontrolle und sind keine Prüfungsbewertung.

Aufgabe	Schwerpunkt	Minuten	Punkte
1	Registerzustände und Aliasnamen	8	8
2	Wortbreite und Bitmasken	9	10
3	Schiebeoperationen	8	8
4	Immediate-Grenzen	7	8
5	Registerformat	13	14
6	Immediate-Format und Adressen	10	12
7	Venus: Gleichung und Vergleich	30	—

Für vier Schulstunden zu je 45 Minuten: 75 Minuten Vorlesung einschließlich Rückfragen, 55 Minuten Papieraufgaben, 30 Minuten Venus, 10 Minuten Quiz und 10 Minuten Auswertung. Das ist eine Planungsannahme; bei größerem Unterstützungsbedarf kann Aufgabe 6 nachbesprochen werden. Die optionale Vertiefung ist zusätzliche Reserve.

Gemeinsame Annahmen

Wir verwenden RV32I, die RISC-V-Integerbasis mit 32-Bit-Registern, ohne komprimierte Erweiterung C. Ein nativer Befehl belegt vier Byte. Adressen zählen Byte, die Kursumgebung ist Little Endian. Registerarithmetik speichert die unteren 32 Bit; `x0` liefert null und verwirft Schreibversuche. Nicht genannte Registerinhalte sind unbekannt und dürfen nicht als null vorausgesetzt werden.

Die ABI, Application Binary Interface, ist die Softwarevereinbarung für Registerrollen. Es gilt `t0=x5`, `t1=x6`, `t2=x7`, `t3=x28`, `t4=x29`, `t5=x30`, `t6=x31`, `a0=x10`, `a1=x11`. Namen bezeichnen Architekturregister; Aliasnamen erzeugen keine weiteren Speicherstellen.

Aufgabe 1: Dieselben Register, neue Werte

8 Punkte · 8 Minuten · Papier

Nur für diese Aufgabe seien `t0`, `t1`, `t2` und `x0` anfangs null. Füllen Sie die Zustände jeweils **nach** der angegebenen Instruktion aus.

```
addi t0, x0, 7
addi t1, x0, 3
add  t0, t0, t1
sub  t1, t0, t1
add  t2, t1, x0
add  x0, t0, t2
```

a) Zustände der sechs Zeilen (4 Punkte). b) Welcher Wert wird in Zeile 3 als erste Quelle gelesen und warum? (1 Punkt). c) Welchen Wert zeigt `x5` nach Zeile 4? Begründen Sie. (1 Punkt). d) Welche Rechnung führt Zeile 6 aus, und was wird gespeichert? (2 Punkte).

nach Zeile	t0	t1	t2	x0
1				
2				
3				
4				
5				
6				

Begründungen b)–d):

Aufgabe 2: Zahlenmuster und Bitmasken

10 Punkte · 9 Minuten · Papier

a) Ergänzen Sie das Ergebniswort mit acht Hexziffern. Lesen Sie dieses Wort zusätzlich als signed Zweierkomplement. Geben Sie an, ob ein Carry über Bit 31 hinaus und ob signed Überlauf entsteht (je 2 Punkte).

Quellen von add	Ergebniswort	signed Wert	Carry	Überlauf
0xFFFFFFFF, 1				
0x7FFFFFFF, 1				
0x80000000, 0x80000000				

b) Es gilt `t0=0x000000A5` und `t1=0x0000000F`. Bestimmen Sie die drei Zielwörter (je 1 Punkt).

```
and t2, t0, t1
or  t3, t0, t1
xor t4, t0, t1
```

c) Warum ist die letzte Operation keine Aussage „die Zahlen sind verschieden, also Ergebnis 1“? (1 Punkt).

Bitrechnung und Begründung:

Aufgabe 3: Weite und Füllbits unterscheiden

8 Punkte · 8 Minuten · Papier

Es gilt `t0=0xFFFFFFFD` (-3), `t1=1`, `t5=32`. Jede Zeile wird unabhängig mit diesen Anfangswerten betrachtet.

a) Bestimmen Sie das Zielwort und begründen Sie die Füllung beziehungsweise effektive Weite (je 1 Punkt).

Instruktion	Zielwort	Füllung / effektive Weite
srl t2, t0, t1		
sra t3, t0, t1		
sll t4, t0, t5		
srai t6, t0, 1		

b) Welches Ergebnis hätte `sll t4, t0, t5` für `t5=33`? (1 Punkt). c) Ist `slli t4, t0, 32` zulässig? Warum unterscheidet sich der Fall von a)? (2 Punkte). d) Warum ist das arithmetische Schieben von -3 um eins nicht gleich einer Division durch zwei mit Abschneiden zur null? (1 Punkt).

Begründungen b)–d):

Aufgabe 4: Konstanten und ihre Grenzen

8 Punkte · 7 Minuten · Papier

a) Entscheiden Sie jeweils: gültige native Instruktion, gültige Pseudoinstruktion oder unzulässiger Operand. Begründen Sie anhand der Breite beziehungsweise des Zahlenbereichs (je 1 Punkt).

Assemblertext	Einordnung und Begründung
addi t0, x0, 2047	
addi t0, x0, 2048	
addi t0, x0, -2048	
li t0, 2048	
andi t0, t1, 4095	
xori t0, t1, -1	

b) Stellen Sie -4 als zwölf Bit, als drei Hexziffern und nach Vorzeichenerweiterung als acht Hexziffern dar (1 Punkt).

c) Wie ersetzen Sie eine Subtraktion von sieben durch `addi`? Warum gelingt die Subtraktion von -2048 nicht mit demselben Schema in einer einzelnen `addi`-Instruktion? (1 Punkt).

Rechnung b) und c):

Aufgabe 5: R-Format in beide Richtungen

14 Punkte · 13 Minuten · Papier

Referenz: Von Bit 31 bis Bit 0 folgen `funct7[31:25]`, `rs2[24:20]`, `rs1[19:15]`, `funct3[14:12]`, `rd[11:7]` und `opcode[6:0]`. Registerfelder enthalten je fünf Bit. Für `add`: opcode `0110011`, funct3 `000`, funct7 `0000000`. Für `sub`: gleicher opcode und funct3, aber funct7 `0100000`.

a) Codieren Sie `sub x7, x5, x6`. Zeigen Sie alle Feldwerte, die zusammengesetzten 32 Bit und das Hexwort (6 Punkte).

b) Decodieren Sie `0x407302B3`. Zeigen Sie die sechs Felder und den Befehl sowohl mit `x`-Namen als auch mit Aliasnamen. Es gilt `t0=x5`, `t1=x6`, `t2=x7` (6 Punkte).

c) In a) wird der Inhalt von `x5` von 7 auf 100 geändert. Ändert sich die Codierung? Genügt opcode `0110011` alleine zum Erkennen von `sub`? Begründen Sie beide Antworten (2 Punkte).

Feldwerte a):

funct7	rs2	rs1	funct3	rd	opcode

32-Bit-Wort, Hexdarstellung und Rückrechnung b):

Begründungen c):

Aufgabe 6: I-Format und Instruktionsadressen

12 Punkte · 10 Minuten · Papier

a) Codieren Sie `addi t2, t1, -7`. Es gilt `t2=x7` und `t1=x6`. Referenz: `imm[31:20]` mit zwölf Bit, `rs1[19:15]`, `funct3[14:12]` = `000`, `rd[11:7]`, `opcode[6:0]` = `0010011`. Zeigen Sie das Zweierkomplement, alle Felder und das Hexwort (6 Punkte).

b) Gegeben ist folgende **native** Lehrentwicklung einer Konstantenladung. Der erste Befehl liegt an Adresse $B = 0x1000$. Notieren Sie die Adressen aller drei Befehle, die Adresse des Labels `Marke` und den PC nach dem dritten Befehl. Wie viele Byte belegen die Befehle insgesamt? (4 Punkte).

```
main:
    lui  t0, 0x1
    addi t0, t0, -2048
Marke:
    addi t1, t0, -7       # letzter Rechenbefehl
```

c) Speichern Sie das Wort aus a) in der Kursumgebung ab Adresse `0x2000`. Geben Sie die vier Byte in aufsteigender Adressreihenfolge an (2 Punkte).

Felder, Binärwort und Hexwort a):

Adressen, PC und Bytezahl b):

Bytefolge c):

Aufgabe 7: Die Gleichung in Venus prüfen

30 Minuten · Rechner und Papierprotokoll

Implementieren Sie $Y = (A + B) - (C + D)$ mit $A = 5$, $B = 10$, $C = 3$, $D = 2$. Belegen Sie `t0`–`t3` mit den Eingaben, `t4` und `t5` mit den beiden Summen und `t6` mit dem Ergebnis. Die Eingaben bleiben erhalten.

1. Sagen Sie die drei Rechenergebnisse voraus, bevor Sie Venus starten.
2. Schreiben Sie ein vollständiges Programm mit `.text`, `.globl main` und Startlabel `main:`. Initialisieren Sie mit `li`.
3. Setzen Sie das Label `Ergebnis_pruefen:` unmittelbar vor die Vorbereitung der Ausgabe. Vergleichen Sie dort die vorhergesagten und angezeigten Registerwerte.
4. Venus: Kopieren Sie `t6` nach `a1`, setzen Sie `a0=1` und führen Sie `ecall` aus. Anschließend setzen Sie `a0=10` und beenden mit `ecall`. Der erste Dienst druckt eine signed Ganzzahl, der zweite beendet das Programm. Diese Vereinbarung ist nicht die normale Funktionsrückgabe in `a0`.
5. Prüfen Sie die drei Varianten unten nach **erneutem Assemblieren und Neustart**. Notieren Sie die erste Abweichung, falls eine auftritt.

Eingaben (A, B, C, D)	erwartete Summen, Y	Beobachtung vor der Ausgabe
$(5, 10, 3, 2)$	selbst vorhersagen	
$(0, 0, 0, 0)$	selbst vorhersagen	
$(5, 1, 10, 2)$	selbst vorhersagen	
$(-5, 10, 3, 2)$	selbst vorhersagen	

Die Schrittanzeige verfolgt native Instruktionen und misst keine Hardwaretakte. Prüfen Sie die tatsächlich erzeugten Instruktionen, insbesondere bei Pseudoinstruktionen. Das Programm soll nach der Ausgabe kontrolliert enden.

Eigener Code und Vergleichsprotokoll:

Optionale Vertiefung: Pseudotext und Beobachtung

Zusätzliche Reserve · nicht in den 85 Übungsminuten enthalten

a) Schreiben Sie ein vollständiges Venus-Programm, das fünf in `t1` lädt, mit einem einzigen `slli` um drei Stellen in `t2` schiebt und das Ergebnis mit `sub` nach `t3` negiert. Prüfen Sie vor dem Abschluss die drei Register. Welche Ergebnisse erwarten Sie?

b) Öffnen Sie anschließend vl2_immediates.s. Sagen Sie `s1` bis `s5` und `s8` vor dem Label `Ergebnis_pruefen` voraus. Die Aliasregister `s0`–`s11` werden hier ausschließlich im Hauptprogramm verwendet; eine aufgerufene Funktion wird nicht modelliert.

c) Untersuchen Sie in der nativen Instruktionsansicht die Entwicklung von `li s4, 2048`. Halten Sie alle erzeugten Befehle sowie den Zwischenwert nach dem ersten davon fest. Vergleichen Sie mit der gültigen Lehrentwicklung `lui s4, 0x1` und `addi s4, s4, -2048`. Beschreiben Sie die tatsächlich beobachtete Folge, auch wenn sie anders dargestellt wird.

Vorhersage, Code und protokollierte Expansion: